MARKET RESEARCH · PRODUCT PRESENTATION

TickerRadar

An AI-powered market research platform — **deterministic scoring** you can audit, plus **AI deep-dive analyses** with verified citations.

PART 1

What the app is & how it works

The numbers behind every score — and the stock terms, in plain English.

PART 2

The Python intelligence module

How it plugs into any Rails app — and what it can do for them.

BONUS

Documentation in-app

A full plain-English Documentation page ships with the product.

PART 1 · THE PRODUCT

What is TickerRadar?

TickerRadar watches the US stock market for you. It gathers prices, company financials, news and events — then turns that raw data into **scores, opportunities and written analyses** you can actually read.

📊 It scores companies

Every company gets an **investment score (0–100)**, a **risk score** and a **confidence level** — computed by fixed, auditable rules. Not a black box.

🤖 It writes the analysis

An AI writes a plain-English thesis per company: why it looks attractive, what the risks are — and **every claim links to a cited source**.

🔔 It watches for you

News, events and score changes flow into **opportunity lists and alerts**, so you don't have to stare at charts all day.

What you can do inside the app

Dashboard
Market pulse and your watchlist at a glance.

Companies
Browse, filter and open a full company deep dive.

Opportunities
Ranked candidates that pass the scoring rules.

Portfolio & Backtest
Build a portfolio and test a strategy against history.

PART 1 · HOW IT WORKS

The three numbers — and what each one means

INVESTMENT SCORE

0–100

"How attractive does this company look **under our rules**?" Higher = more attractive. It is a **research ranking, not a predicted return**.

RISK SCORE

0–100

"How much concern do we see?" Higher = more concern. **Read it separately** — a good score with a high risk score is a trade-off, not a green light.

CONFIDENCE

0–1

"How much real data backs this analysis?" Low confidence means **inputs were thin or missing** — not that the conclusion is wrong.

Common trap: a score of `50` can mean "we lack information" — missing inputs fall back to a neutral 50 with lower confidence. Always read **coverage & warnings** alongside the score.

PART 1 · THE NUMBERS

How the investment score is calculated — and why

A weighted average of **seven components**, computed by fixed Python rules (never the AI model), with risk subtracted. Weights are configurable — these are the defaults:

Component	Weight	Why it's in there
Fundamentals	30%	Business health: revenue, margins, debt, cash flow.
Valuation	20%	Price vs. earnings & book value — cheap or expensive?
Growth	15%	Revenue & earnings trend over time.
Technical	10%	Price trends & momentum.
Sentiment	10%	Tone of recent news coverage.
Catalysts	10%	Recent / upcoming events that could move the price.
Risk	−15%	Volatility, leverage, downside — **subtracted**.

Worked example (defaults)

If all six positive components score 60 and risk scores 40:

```
score = 60 × (0.30+0.20+0.15+0.10+0.10+0.10)
      − 40 × 0.15
      = 60 × 0.95 − 6 = 51 → NEUTRAL
```

Score → recommendation label

80–100	**STRONG_OPPORTUNITY**
70–79	**OPPORTUNITY**
60–69	**WATCH**
45–59	**NEUTRAL**
30–44	**CAUTION**
15–29	**HIGH_RISK**
< 15	**AVOID**

Labels are app categories under our rules — not trade instructions.

Why rules, not AI, for the score? Deterministic = the same inputs and rules always reproduce the same number. You can audit it, backtest it, and tune the weights. The AI never invents the score.

PART 1 · THE NUMBERS

What goes into the calculation

Raw data ingested per company

- **Market prices** — OHLCV (Open, High, Low, Close, Volume), adjusted for splits.
- **Fundamentals** — revenue, profit margins, debt, cash flow, valuation ratios.
- **Technical indicators** — moving averages, momentum, trend signals.
- **News** — collected, classified and scored for positive/negative tone.
- **Events** — earnings dates and other scheduled catalysts.
- **Macro context** — market-level conditions for situational awareness.

Missing data policy

A data-quality check **weighs each input before scoring** (price 30% · fundamentals 30% · technicals 15% · news 15% · events 10%). If an input is missing or too thin it is **skipped** — the score falls back to a neutral 50 for that part, with **lower confidence**. We never fill gaps with guesses.

Plain-English glossary (terms you'll see in the app)

Ticker	Short identifier for a stock (e.g. `AAPL` = Apple).
Market cap	Share price × number of shares — company size, not cash balance.
P/E ratio	Price ÷ earnings per share. High can mean expensive *or* high growth expectations.
P/B ratio	Price ÷ book value (assets minus liabilities). Compares price to net worth.
EPS	Earnings per share — the company's profit split across each share.
Dividend	Cash the company pays shareholders; price-only returns may leave it out.
Volume	Shares traded. High volume = lots of trading, not necessarily buyers beating sellers.
SMA / EMA	Average price over N days (simple / weighted to recent). Used to spot trends.
RSI	0–100 momentum gauge: high = possibly overbought, low = possibly oversold.
Volatility	How much the price swings. Higher = bigger up *and* down moves.
Beta	How strongly a stock moves with the market. ≈1 means similar sensitivity.
Benchmark	Reference used to compare performance (e.g. an index or ETF).
Alpha	Strategy return minus benchmark return over the same period.

Full glossary ships in-app: **Documentation** page + `STOCK_TERMS_IN_PLAIN_ENGLISH.md`.

PART 1 · WRAP-UP

The AI analysis pipeline — in five steps

1
Collect
Prices, fundamentals, news & events ingested per company.

→

2
Score (rules)
Deterministic Python engine produces the numbers — no AI involved.

→

3
Retrieve (RAG)
Relevant news & documents found by semantic similarity.

→

4
Write (AI)
LLM drafts thesis, risks & recommendation citing the evidence.

→

5
Verify
Every claim must map to a registered source; unsupported citations are dropped.

How to read a deep dive

- Check **confidence & data coverage** first.
- Read **investment score** and **risk score** together.
- Follow the **citations** — the AI links to sources; unverified claims are removed before you see them.

Know the limitations

- Scores are a research ranking — **not investment advice** or a return forecast.
- Scores are point-in-time: data and rules change, older scores age.
- AI-written text can contain errors — always check cited sources.
- Confidence = input completeness, not probability of being right.

PART 2 · THE PLATFORM BEHIND THE PRODUCT

The Python intelligence module, reusable in any Rails app

One rule drives the whole design: **“The framework owns intelligence mechanics. A domain owns domain meaning.”**

WHAT IT IS

A standalone **FastAPI service** with a reusable intelligence engine (scoring, RAG, embeddings, LLM transport, validation, scheduling) and pluggable **domain packs** — `stock` ships today, `hr` proves the pattern.

WHAT IT GIVES YOU

Drop it next to any Rails monolith and get **AI analysis, scored entities, evidence-backed narratives and background pipelines** — without rebuilding any of the intelligence plumbing.

PART 2 · ARCHITECTURE

How it plugs into your Rails monolith



Contract: Rails calls the HTTP API with HMAC-signed requests (service token + user context). The platform never touches Rails' database — it has its own Postgres schema with **entity UUID references** back to Rails. No cross-database joins.

PART 2 · THE ENGINE

What's inside — and why it's built this way

Core framework (framework-owned)

Piece	What it does
Scoring engine	Weighted, deterministic, versioned. Domain packs supply weights.
RAG pipeline	Chunking → embedding (pgvector) → retrieval → prompt assembly.
LLM transport	OpenAI-compatible clients, retries, timeouts, cost controls, JSON-mode.
Validation & persistence	Schemas, idempotency keys, audit trail of every run.
Scheduler & queues	Celery/Redis pipelines for ingest, scoring and narrative refresh.
Observability	Structured logs, run metrics, token/cost tracking.

Domain pack (domain-owned)

- **Entity types** — what is being scored (e.g. company, candidate).
- **Feature extractors** — turn raw inputs into named features.
- **Weights & rule presets** — what “good” means in this domain.
- **Prompt templates** — how the AI is asked to reason here.
- **Pipelines** — ingest → score → narrative orchestration.
- **API routes** — the endpoints Rails calls.

Why two packs? `hr` (recruiting) reuses 100% of the core — scoring, RAG, validation — with completely different features, weights and prompts. That's the proof the framework/domain split holds.

PART 2 · INTEGRATION

Five steps to integrate

1 Deploy the platform

```
git clone …/intelligence-platform
docker compose up -d        # FastAPI + Postgres(pgvector) + Redis
bin/migrate.sh
python -m scripts.smoke     # health check ✓
```

Config via `.env`: DB, Redis, LLM keys, HMAC secrets. Runs on CPU-only; LLM calls are remote.

2 Register the domain

Add entities, features, weights and prompt templates to the `stock` pack (or copy the pack skeleton into a new one). Register sources so every AI claim can be traced to an origin.

3 Point Rails at the API

```
# config/application.rb or initializer
INTELLIGENCE_PLATFORM_URL = ENV["IP_URL"]
INTELLIGENCE_PLATFORM_TOKEN = ENV["IP_HMAC_TOKEN"]
```

Rails signs each request (service token + user id). The platform resolves the user context but never reads Rails' DB.

4 Call the intelligence endpoints

```
POST /api/v1/analyze
{
  "entity_id": "c9f1…",        # Rails-side UUID
  "entity_type": "company",
  "include": ["score", "narrative", "risks"]
}
→ 202 Accepted (pipeline enqueued)
GET /api/v1/analysis/:id
→ { "investment_score": 74, "confidence": 0.82,
    "narrative": { "thesis": "…", "citations": [...] } }
```

5 Run pipelines & observe

```
celery -A intelligence.tasks worker
celery -A intelligence.tasks beat   # scheduled refresh
```

Nightly ingest → score → narrative refresh. Every run is logged with duration, token cost and validation status. Rails displays results verbatim — **no intelligence logic lives in Rails**.

> **Time to first value:** deploy (~30 min) → register domain (existing `stock` pack works as-is) → first scored entity within one pipeline cycle.

PART 2 · VALUE

What it can do for other Rails apps

Any Rails product that needs to **turn data into scores, narratives and alerts** can reuse the engine. The `hr` domain already proves it works outside finance:

PROVEN · SHIPPING

📈 Stock intelligence

Investment/risk scoring, deep dives with citations, opportunity ranking, alerts, portfolios & backtesting.

PROVEN · SAME ENGINE

👥 HR / recruiting

Candidate **fit scores**, employee-review sentiment buckets, evidence-backed candidate summaries.

READY TO BUILD

📄 Document intelligence

Ingest documents → embeddings → RAG Q&A and structured summaries with provenance, for any document-heavy app.

READY TO BUILD

🛒 E-commerce / catalogs

Product scoring, review sentiment analysis, AI-generated buying guides with cited sources.

READY TO BUILD

🏢 Real estate, travel, ops…

Any domain with entities + events + text: listing scores, event detection, automated briefings.

PLATFORM-LEVEL

⚙️ Cross-cutting wins

Deterministic scoring you can audit & backtest · citation verification · data-quality-aware confidence · scheduled pipelines · multi-provider abstraction.

The pitch in one sentence: you bring the domain data and the meaning; the module brings the entire intelligence machinery — scoring, RAG, LLM, validation, scheduling — as a service.

PART 2 · PRODUCTION-READY

Security & operations

🔐 Security model

- **HMAC-signed requests** — service token proves Rails' identity; no user JWTs cross the boundary.
- **User scoping** — user context travels with each request so per-user analysis stays isolated.
- **No shared database** — UUID references only; no cross-DB joins or leakage paths.
- **Least privilege** — the platform needs API access to LLMs and its own DB, nothing else.

🛠️ Operations

- **Failure isolation** — LLM outage degrades narratives; deterministic scores keep flowing.
- **Observability** — every run logged with duration, token cost, validation status.
- **Idempotency** — retries are safe; duplicate analysis requests collapse to one run.
- **Horizontal scaling** — API and workers scale independently of Rails.

Zero-risk trial: run the platform in parallel for one domain feature (e.g. narratives only) while Rails keeps its existing logic. Compare outputs, then flip the feature over.

SUMMARY

Key takeaways

PART 1 — THE PRODUCT

- TickerRadar = **data in → scored, cited, plain-English analysis out**.
- Three numbers: **investment score, risk score, confidence** — read together, with coverage.
- Scores come from **fixed, weighted rules** (auditable, backtestable) — never from the AI.
- The AI only **writes the narrative**, from retrieved evidence, with verified citations.
- Full glossary & methodology ship **in-app under Documentation**.

PART 2 — THE PLATFORM

- **Framework owns mechanics, domain owns meaning** — the engine is domain-agnostic.
- **HMAC-authenticated FastAPI service + 5-step integration** = full intelligence stack in any Rails app.
- Proven in two domains (**stock** & **HR**); extensible to documents, e-commerce, real estate…
- Secure by design: signed requests, per-user scoping, no shared DB, idempotency, failure isolation.

Thank you — questions?